Day 7 Solutions:

Question 1: *Presents*

Solution: You are given an integer n representing the number of friends.
Each friend gives **exactly one present** to another friend.

An array a of size n is given where:

- a[i] denotes the **friend number who received a present from friend (i + 1)**.

Your task is to determine, for every friend i, **who gave them a present**.

Key Observation

- Friend i + 1 gives a present to friend a[i]
- Friend a[i] received a present from friend i + 1

So we need to **reverse the mapping**.

Approach

1. Read array a where:
    - Index i → giver (i + 1)
    - Value a[i] → receiver
2. Create another array b of size n such that:
    - b[x] stores **who gave a present to friend (x + 1)**
    - for each index i, place i + 1 at position a[i] - 1 in b
3. Print array b

Time Complexity: O(n) & **Space Complexity:** O(n)

Both are optimal for this problem.

```cpp
#include<bits/stdc++.h>
#define v vector<long long>
#define vv vector<v>
#define vvv vector<vv>
#define ll long long
#define cyes cout<<"YES\n"
#define cno cout<<"NO\n"
using namespace std;
int main(){
  cin.tie(0);cout.tie(0);ios::sync_with_stdio(0);
  ll n;cin>>n;
  v a(n),b(n);
  for(ll &x:a)cin>>x;
  for(ll i=0;i<n;i++)b[a[i]-1]=i+1;
  for(ll x:b)cout<<x<<" ";
}
```

Question 2: Lucky Division

Solution: A number is called **lucky** if it consists only of digits **4** and **7**.

Given an integer n, determine whether n is **divisible by at least one lucky number**.

If yes, print YES, otherwise print NO.

Key Insight

The maximum value of n is small (n ≤ 1000)

All possible lucky divisors within this range are limited and known

Lucky Numbers Considered

The code explicitly checks divisibility by the following lucky numbers:

4, 7, 47, 74, 447, 474, 477, 747, 774 (also 44, 77, 444, 744, and 777 but if someone is divisible by these then it is already divisible by 4 or 7)

If n is divisible by **any one** of them, the answer is YES.

Time and Space Complexity

- **Time Complexity:** O(1)
- **Space Complexity:** O(1)

The solution uses a constant number of operations.

```
#include<bits/stdc++.h>
#define v vector<long long>
#define vv vector<v>
#define vvv vector<vv>
#define ll long long
#define cyes cout<<"YES\n"
#define cno cout<<"NO\n"
using namespace std;
int main(){
  cin.tie(0);cout.tie(0);ios::sync_with_stdio(0);
  ll n;cin>>n;
  if((n%4==0)||(n%7==0)||(n%47==0)||(n%74==0)||(n%447==0)||(n%474==0)||(n%477==0)||(n%747==0)||(n%774==0))cyes;
  else cno;
}
```

Question 3: Boxes Packing

Answer:

Key Observation

Boxes of the same size **must be placed in different piles** (because each pile can only contain one box of size less than the size of itself).Therefore, the **maximum frequency** of any box size determines the minimum number of piles needed.

Answer = maximum number of times any size appears in the array

Approach

1. Sort the array of size of boxes
2. Traverse and track the **maximum frequency**
3. Output this maximum value

Time and Space Complexity

- **Time Complexity:** O(n log n) due to sorting
- **Space Complexity:** O(n) for storing the array

```cpp
#include<bits/stdc++.h>
#define v vector<long long>
#define vv vector<v>
#define vvv vector<vv>
#define ll long long
#define cyes cout<<"YES\n"
#define cno cout<<"NO\n"
using namespace std;
int main(){
  cin.tie(0);cout.tie(0);ios::sync_with_stdio(0);
  ll n,cnt=0,ans=0,curr=0;cin>>n;
  v a(n);
  for(ll &x:a)cin>>x;
  sort(a.begin(),a.end());
  for(ll x:a){
    if(x!=curr){
      curr=x;
      cnt=0;
    }
    cnt++;
    ans=max(ans,cnt);
  }
  cout<<ans;
}
```

Bonus Question: Boboniu Plays Chess

Solution:

Problem Summary

You are given a chessboard of size n × m and a starting cell (x, y). Your task is to output a **sequence of moves** such that:

Every cell of the board is visited **exactly once**. The starting cell (x, y) is visited **first.** You only need to output **one valid traversal**.

Core Idea

This problem does **not** require shortest paths or chess rules. It only asks for **any path** that:

Starts at (x, y). Visits all cells exactly once.

A **snake-like traversal (zig-zag pattern)** across columns is sufficient. (Because observe that you are not at border and you can jump the visited cell to visit an unvisited just be care full that you don't output the same cell again).

Strategy Used in the Code

1. Start at (x, y)
2. Move to (x, 1)
3. Move to (1, 1)
4. Traverse the entire grid column by column:
    - Odd columns → top to bottom
    - Even columns → bottom to top
5. Skip cells already printed

Initial Mandatory Moves

```
cout << x << ' ' << y << endl;

cout << x << ' ' << 1 << endl;

cout << 1 << ' ' << 1 << endl;
```

Main Traversal Logic

```
for(ll i = 1; i <= m; i++){
```

Loop through each column.

Odd Columns (Top to Bottom)

```
if(i % 2 == 1){

for(ll j = 1; j <= n; j++){
```

Traverse rows from top to bottom.

```
if((j!=x || i!=y)&&(j!=x || i!=1)&&(j!=1 || i!=1))

cout << j << ' ' << i << endl;
```

Skip already printed cells:

- (x, y)
- (x, 1)
- (1, 1)

Even Columns (Bottom to Top)

else{

for(ll j = n; j >= 1; j--){

Traverse rows from bottom to top.

if((j!=x || i!=y))

cout << j << ' ' << i << endl;

Only (x, y) needs to be skipped here.

Time and Space Complexity

- **Time Complexity:** O(n × m)
- **Space Complexity:** O(1) (no extra data structures)

```
#include<bits/stdc++.h>
#define v vector<long long>
#define vv vector<v>
#define vvv vector<vv>
#define ll long long
#define cyes cout<<"YES\n"
#define cno cout<<"NO\n"
using namespace std;
int main(){
  cin.tie(0);cout.tie(0);ios::sync_with_stdio(0);
  ll n,m,x,y;cin>>n>>m>>x>>y;
  cout<<x<<' '<<y<<endl<<x<<' '<<1<<endl<<1<<' '<<1<<endl;
  for(ll i=1;i<=m;i++){
    if(i%2==1){
      for(ll j=1;j<=n;j++){
        if((j!=x || i!=y)&&(j!=x || i!=1)&&(j!=1 || i!=1))cout<<j<<' '<<i<<endl;
      }
    }
    else{
      for(ll j=n;j>=1;j--){
        if((j!=x || i!=y))cout<<j<<' '<<i<<endl;
      }
    }
  }
}
```